12

SECURING YOUR NEST EGG

Baby boomers are Americans born between 1946 and 1964. They form a large demographic cohort—about 20 percent of the American population—so they've had a huge influence on all aspects of American culture. The financial industry was quick to cater to their needs, which for decades concerned investment growth. But in 2011, the oldest boomers reached age 65 and began retiring in droves, at the rate of 10,000 *per day*! With an average life span longer than that of preceding generations, a boomer may enjoy a retirement as long as their career. Funding this 30- to 40-year period is a big problem and a big business.

In the years when financial advisers were mainly focused on *growing* boomers' wealth, they relied on the simple "4 Percent Rule" for retirement planning. Simply stated, for every year you're retired, if you spend an amount no larger than 4 percent of the savings you had *in the first year of retirement*, you'll never run out of money. But as Mark Twain observed, "All generalizations are false, including this one!" The value of our investments and the amount we spend are constantly in flux, often due to forces outside of our control.

As a more sophisticated alternative to the 4 Percent Rule, the financial industry adopted Monte Carlo simulation (see **Chapter 11** for an overview of MCS). With MCS, you can test and compare retirement strategies over

thousands of lifetimes. The goal is to identify how much money you can spend each year in retirement, given your life expectancy, without exhausting your savings.

The advantage of MCS over other methods increases as the sources of uncertainty increase. In **Chapter 11**, you applied MCS to a single variable with a simple probability distribution. Here you'll look at uncertainty around life spans, while capturing the true cyclicity and interdependency of the stock and bond markets and inflation. This will allow you to evaluate and compare different strategies for achieving a secure and happy retirement.

Project #20: Simulating Retirement Lifetimes

If you think you're too young to worry about retirement, think again. The baby boomers thought the same thing, and now over half of them have insufficient savings for retirement. For most people, the difference between eating Kobe beef or dog food in retirement is how soon they start saving. Due to the magic of compounding interest, even modest savings can add up over decades. Knowing early the numbers you'll need later enables you to set realistic goals for a painless transition to your golden years.

THE OBJECTIVE

Build a Monte Carlo simulation for estimating the probability of running out of money in retirement. Treat years in retirement as a key uncertainty and use historical stock, bond, and inflation data to capture the observed cyclicity and dependency of these variables.

The Strategy

To plan your project, don't hesitate to check out the competition. Numerous nest-egg calculators are available online for free. If you play with these, you'll see that they display a high level of input-parameter variability.

Calculators with many parameters may seem better (see **Figure 12-1**), but with each added detail, you start going down rabbit holes, especially when it comes to the US's complex tax code. When you're predicting outcomes 30 to 40 years into the future, details can become noise. So, you're better off keeping it simple, focusing on the most important and controllable issues. You can control when you retire, your investment asset allocation, how much you save, and how much you spend, but you can't control the stock market, interest rates, and inflation.

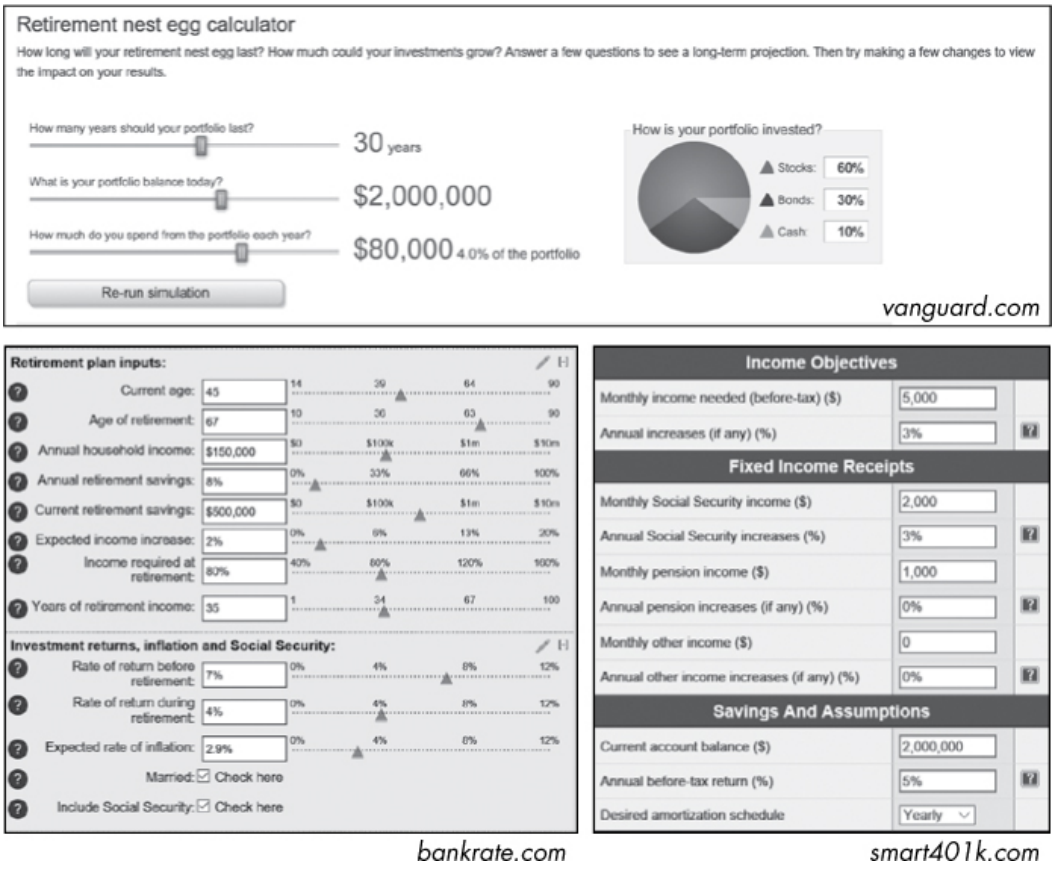

Figure 12-1: Example input panels for three online nest-egg calculators

When you can't know the "correct" answer to a problem, looking at a range of scenarios and making decisions based on probabilities is best. For decisions that involve a "fatal error," like running out of money, desirable solutions are those that lower the likelihood of that event.

Before you get started, you need to speak the language, so I put together a list of the financial terms you'll use in this project:

Bonds A bond is debt investment in which you loan money to an entity—usually a government or corporation—for a set length of time. The borrower pays you an agreed-upon rate of interest (the bond's *yield*) and, at

the end of the term, returns the full value of the loan, assuming the issuing entity doesn't go broke and default. The value of bonds can go up and down over time, so you may lose money if you sell a bond early. Bonds are appealing to retirees, as they offer safe, steady, predictable returns. Treasury bonds, issued by the US government, are considered the safest of all. Unfortunately, most bond returns tend to be low, so they are vulnerable to inflation.

Effective tax rate This is the average rate at which an individual or married couple is taxed. It includes local, state, and federal taxes. Taxes can be complicated, with large variances in state and local tax rates, many opportunities for deductions and adjustments, and variable rates for different types of income (such as short- versus long-term capital gains). The tax code is also *progressive*, which means you pay proportionally more as your income increases. According to The Motley Fool, a financial services company, the average American's overall income-based tax rate was 29.8 percent in 2015. And this excludes sales and property taxes! You can also count on Congress to tinker with these rates at least once in a 30-year retirement. Due to these complexities, for this project you should adjust your *withdrawal* (spending) parameter to account for taxes.

Index It is safest to invest in many assets, rather than putting all your (nest) eggs in one basket. An index is a hypothetical portfolio of securities, or group of baskets, designed to represent a broad part of a financial market. The Standard & Poor (S&P) 500, for example, represents the 500 largest US companies, which are mostly dividend-paying companies. Index-based investments—such as an index mutual fund—allow an investor to conveniently buy one thing that contains the stocks of hundreds of companies.

Inflation This is the increase in prices over time due to increasing demand, currency devaluation, rising energy costs, and so on. Inflation is an insidious destroyer of wealth. The inflation rate is variable, but has averaged about 3 percent annually since 1926. At that rate, the value of money is halved every 24 years. A little inflation (1 to 3 percent) generally indicates that the economy is growing and wages are going up. Higher inflation and negative inflation are both undesirable.

Number of cases These are the trials or runs performed during MCS; each case represents a single retirement lifetime and is simulated with a new set of randomly selected values. For the simulations you will be running, somewhere between 50,000 and 100,000 cases should provide a suitably repeatable answer.

Probability of ruin This is the probability of running out of money before the end of retirement. You can calculate it as the number of cases that end with no money divided by the total number of cases.

Start value The starting value is the total value of liquid investments, including checking accounts, brokerage accounts, tax-deferred Individual Retirement Accounts (IRAs), and so on, that are held at the start of retirement. It doesn't equate to *net worth*, which includes assets like houses, cars, and Fabergé eggs.

Stocks A stock is a security that signifies ownership in a corporation and represents a claim on part of the corporation's assets and earnings. Many stocks pay *dividends*, a regular payment similar to the interest paid by a bond or a bank account. For the average person, stocks are the quickest way to grow wealth, but they're not without risk. The price of a stock can go up and down rapidly over a short time—both because of the company's performance and because of speculation arising from investor greed or fear. Retirees tend to invest in the largest dividend-paying US companies because they offer regular income and a less volatile stock price than do smaller companies.

Total returns The sum of capital gains (changes in asset value, like stock price), interest, and dividends is considered the total return of an investment. It is usually quoted on an annual basis.

Withdrawal Also referred to as costs or spending, withdrawal is the pretax, gross income you'll need to cover all expenses in a given year. For the 4 Percent Rule, this would represent 4 percent of the start value in the first year of retirement. This number should be adjusted for inflation in each subsequent year.

Historical Returns Matter

Nest-egg simulators that use constant values for investment returns and inflation (see **Figure 12-1**) grossly distort reality. Predictive ability is only as good as the underlying assumptions, and returns can be highly volatile, interdependent, and cyclical. This volatility impacts retirees the most when the start of retirement—or a large unexpected expense—coincides with the beginning of a big market downturn.

The chart in **Figure 12-2** displays the annual returns for both the S&P 500 index of the largest US companies and the 10-year Treasury bond, which is a reasonably safe, medium-risk, fixed-income investment. It also includes annual inflation rates and significant financial events like the Great Depression.

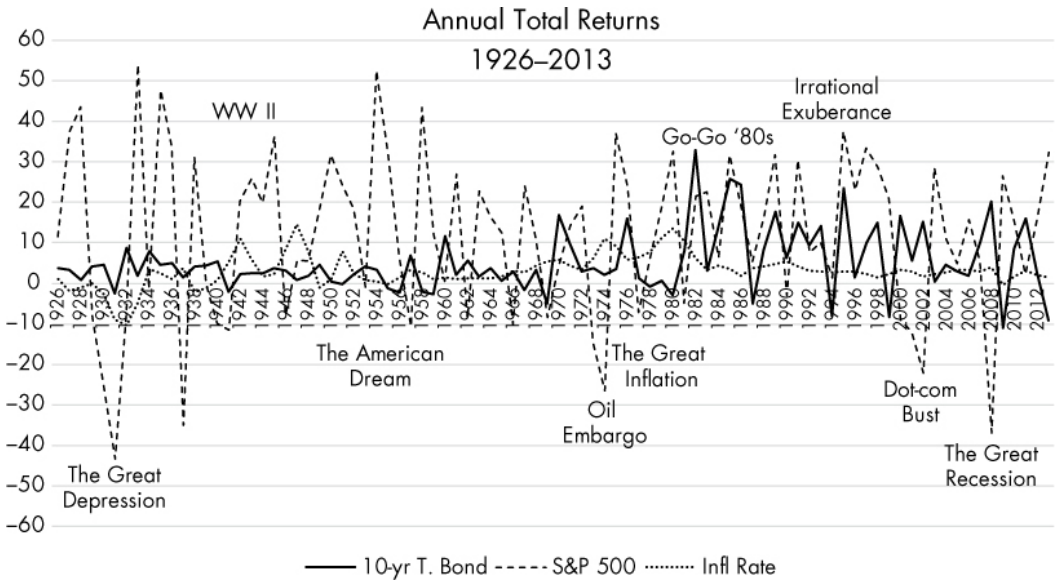

Figure 12-2: Annual rate of inflation plus total returns for stock and bond markets, 1926 to 2013

Financial scholars' long study of the trends in **Figure 12-2** have led to some useful observations about US markets:

- Upward-moving (bull) markets tend to last five times as long as downward-moving (bear) markets.
- Harmful high inflation rates can persist for as long as a decade.
- Bonds tend to deliver low returns that struggle to keep up with inflation.
- Stock returns outpace inflation easily, but at the cost of great volatility in prices.
- Stock and bond returns are often inversely correlated; this means that bond returns decrease as stock returns increase, and vice versa.

- Neither the stocks of large companies nor Treasury bonds can guarantee you a smooth ride.

Based on this information, financial advisers recommend that most retirees hold a diversified portfolio that includes multiple investment types. This strategy uses one investment type as a "hedge" against another, dampening the highs but raising the lows, theoretically reducing volatility.

In **Figure 12-3**, annual investment returns are plotted using both the S&P 500 and a hypothetical 40/50/10 percent blend of, respectively, the S&P 500, the 10-year Treasury bond, and cash. The three-month Treasury bill, which is a very short-term bond with a stable price and low yield (like money stuffed in your mattress), represents cash.

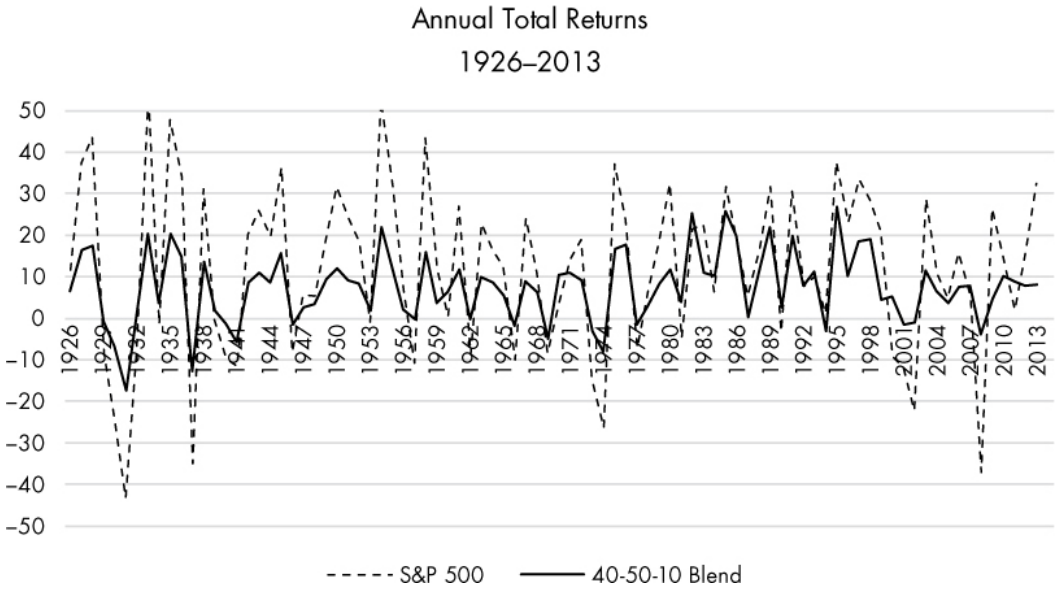

Figure 12-3: Annual returns of the S&P 500 versus a blend of the S&P 500, 10-year Treasury bond, and cash, 1926 to 2013

This diversified portfolio provides a smoother ride than the stock market alone, while still providing protection from inflation. But it will clearly produce different results than online calculators that assume returns are *always* constant and positive in value.

By using historical data, you capture the true *measured* duration of good times and bad, along with the highest highs and the lowest lows. You also account for something completely ignored by the 4 Percent Rule: *the black swan*.

Black swans are consequential, improbable events. These can be good, like meeting your spouse, or bad, like Black Monday, the stock market crash of October 1987. An advantage of MCS is that it can factor in these unexpected events; a disadvantage is that you have to program them, and if they are truly unforeseeable, how can you know what to include?

Black swans that have already occurred, like the Great Depression, are captured in the annual values in lists of historical returns. So, a common approach is to use historical results and assume that nothing worse—or better—will occur in the future. When a simulation uses data from the Great Depression, the modeled portfolio will experience the same stock, bond, and inflation behavior as real portfolios at the time.

If using past data seems too restrictive, you can always edit the past results to reflect lower lows and higher highs. But most people are practical and are happier dealing with events they *know* have occurred—as opposed to zombie apocalypses or alien invasions—so true historical results offer a *credible* way to inject reality into financial planning.

Some economists argue that inflation and returns data before 1980 are of limited use, because the Federal Reserve now takes a more active role in monetary policy and the control of inflation. On the other hand, that is *exactly* the kind of thinking that leaves us exposed to black swans!

The Greatest Uncertainty

The greatest uncertainty in retirement planning is the date you—or your surviving spouse—die, euphemistically referred to as "end of plan" by financial advisers. This uncertainty affects every retirement-related decision, such as when you retire, how much you spend in retirement, when you start taking Social Security, how much you leave your heirs, and so on.

Insurance companies and the government deal with this uncertainty with *actuarial life tables*. Based on the mortality experience of a population, actuarial life tables predict the life expectancy at a given age, expressed as the average remaining number of years expected prior to death. You can find the Social Security table at **https://www.ssa.gov/oact/STATS/table4c6.html**. Based on this table, a 60-

year-old woman in 2014 would have a life expectancy of 24.48 years; this means end of plan would occur during her 84th year.

Actuarial tables work fine for large populations, but for individuals, they're just a starting point. You should examine a range of values, tailored to family history and personal health issues, when preparing your own retirement plan.

To handle this uncertainty in your simulation, consider years in retirement a *random variable*, whose values are chosen at random from a frequency distribution. For example, you can enter the most likely, minimum, and maximum number of years you expect to be retired and use these values to build a triangular distribution. The most-likely value can come from an actuarial table, but the endpoints should be based on your personal health outlook and family history.

An example outcome, based on a triangular distribution for years in retirement for a 60-year-old man, is shown in **Figure 12-4**. The minimum retirement period was set to 20 years, the most likely to 22 years, and the maximum to 40 years. The number of draws from the distribution was 1,000.

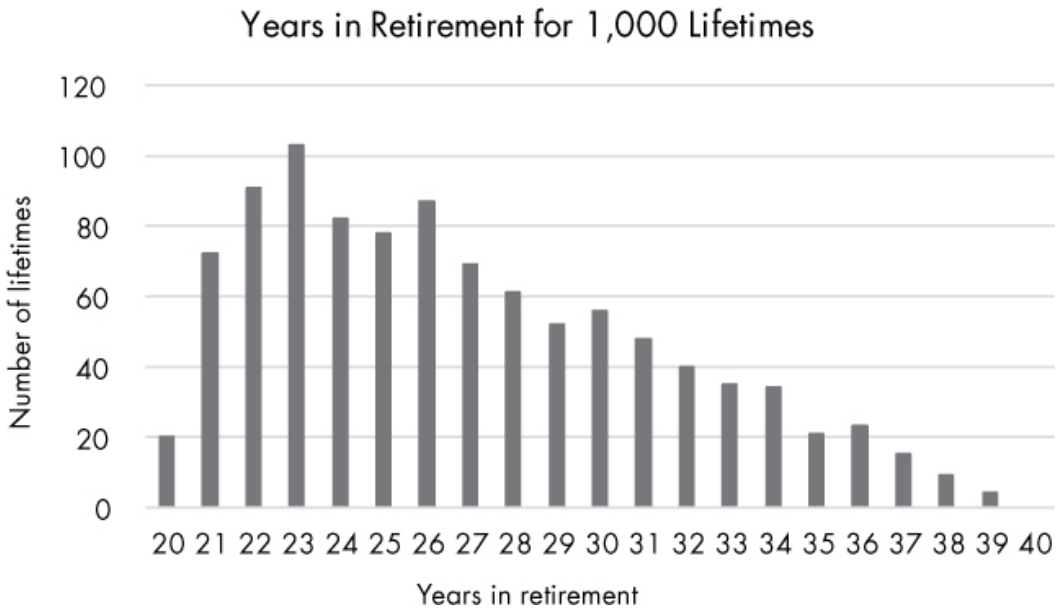

Figure 12-4: Number of lifetimes versus years in retirement based on 1,000 draws from a triangular distribution

As you can see, every possible time interval between the minimum and maximum values is available for simulation, but the intervals taper off in

frequency from the most likely value to the maximum value, indicating that living to 100 is possible but unlikely. Note also that the plot is significantly skewed to the high side. This will ensure conservative results, since—from a financial perspective—dying early is an optimistic outcome and living longer than expected represents the greatest financial risk.

A Qualitative Way to Present Results

An issue with MCS is making sense of thousands of runs and presenting the results in an easily digestible manner. Most online calculators present outcomes using a chart like the one in **Figure 12-5**. In this example, for a 10,000-run simulation, the calculator plots a few selected outcomes with age on the x-axis and investment value on the y-axis. The curves converge on the left at the starting value of the investments at retirement, and they end on the right with their value at end of plan. The overall probability of the money's lasting through retirement may also be presented. Financial advisers consider probabilities below 80 to 90 percent risky.

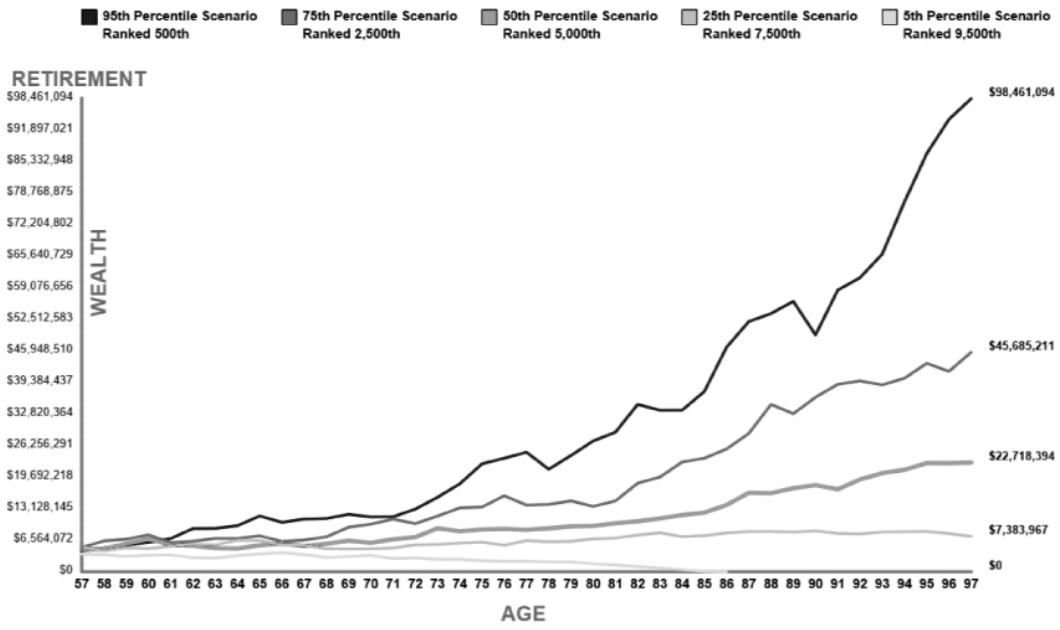

Figure 12-5: Example display from a typical financial industry retirement simulator

The most important piece of information from this type of analysis is the probability of running out of money. It is also interesting to see the endpoint and average outcomes and a summary of the input parameters. In your Python simulator, you can print these in the interpreter window, as shown here:

```
Investment type: bonds
Starting value: $1,000,000
Annual withdrawal: $40,000
Years in retirement (min-ml-max): 17-25-40
Number of runs: 20,000

Odds of running out of money: 36.1%

Average outcome: $883,843
Minimum outcome: $0
Maximum outcome: $7,607,789
```

For a graphical presentation, rather than duplicate what others have done, let's find a new way to present the outcomes. A subset of the results of each case—that is, the money remaining at the end of retirement—can be presented as a vertical line in a bar chart, as in **Figure 12-6**.

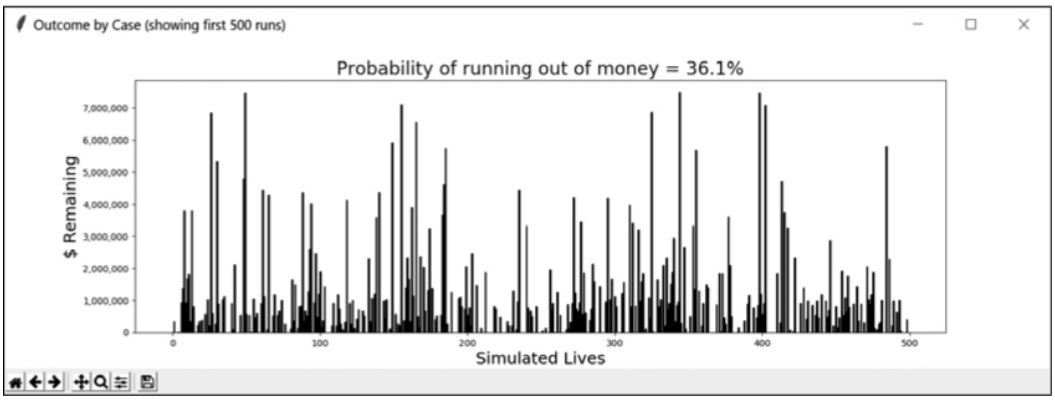

Figure 12-6: The outcomes of simulated retirement periods displayed as vertical columns in a bar chart

In this chart, each bar represents the retirement portion of a single simulated lifetime, and the height of each bar represents the money remaining at the end of that life. Because each bar represents an individual category rather than intervals for continuous measurement, you can arrange the bars in any order without affecting the data. Gaps, representing cases that ran out of money, can be included in the order they occurred in the simulation. With the quantitative statistics recorded in the interpreter window, this diagram provides a qualitative way to present the results.

The peaks and valleys of this chart represent the changing fortunes of many possible futures. In one lifetime, you can die destitute; in the next, you're a multimillionaire. It's evocative of the old saying "There but for the grace of God go I." But, on the other hand, it reinforces General Eisenhower's observation that "plans are useless, but planning is indispensable." Through financial planning, you can "raise the valleys" in the chart and eliminate or greatly reduce your odds of going broke in retirement.

To make this chart, you'll use `matplotlib`, a library that supports 2D plotting and rudimentary 3D plotting. To learn more about `matplotlib` and how to install it, see "**The Probability-of-Detection Code**" on **page 194**.

The Pseudocode

Based on the preceding discussion, the program design strategy should be to focus on a few important retirement parameters and simulate outcomes using the historical behavior of the financial markets. Here's the high-level pseudocode:

```
Get user input for investment type (all stocks, all bonds, or a blend)
Map investment type choice to a list of historical returns
Get user input for the starting value of investments
Get user input for the initial yearly withdrawal amount
Get user input for the minimum, most likely, and maximum duration of retirement
Get user input on number of cases to run
Start list to hold outcomes
Loop through cases:
    For each case:
        Extract random contiguous sample of returns list for duration period
        Extract same interval from inflation list
        For each year in sample:
            If year not equal to year 1:
                Adjust withdrawal for inflation
            Subtract withdrawal from investments
            Adjust investments for returns
            If investments <= 0:
                Investments = 0
```

```
                Break
            Append investments value to outcomes list
    Display input parameters
    Calculate and display the probability of ruin
    Calculate and display statistics
    Display a subset of outcomes as a bar chart
```

Finding Historical Data

You can find returns and inflation information on numerous websites (see "**Further Reading**" on **page 263** for some examples), but I have already compiled the information you need as a series of downloadable text files. If you choose to compile your own lists, be aware that estimates of both inflation and returns may vary slightly from site to site.

For returns, I used three investment vehicles: the S&P 500 stock index, the 10-year Treasury bond, and the three-month Treasury bill, all for the period 1926 to 2013 (1926–1927 values for the Treasury bill are estimates). I used this data to generate additional blended returns for the same period. The following list describes the filenames and contents:

SP500_returns_1926-2013_pct.txt Total returns for the S&P 500 (1926–2013)

10-yr_TBond_returns_1926-2013_pct.txt Total returns for the 10-year Treasury bond (1926–2013)

3_mo_TBill_rate_1926-2013_pct.txt Rate of three-month Treasury bill (1926–2013)

S-B_blend_1926-2013_pct.txt 50/50 percent mix of S&P 500 and 10-year Treasury bond (1926–2013)

S-B-C_blend_1926-2013_pct.txt 40/50/10 percent mix of S&P 500, 10-year Treasury bond, three-month Treasury bill (1926–2013)

annual_infl_rate_1926-2013_pct.txt Average annual US inflation rate (1926–2013)

Here's an example of the first seven lines of the S&P 500 text file:

```
11.6
37.5
43.8
-8.3
-25.1
-43.8
-8.6
```

The values are percentages, but you'll change them to decimal values when you load them in the code. Note that years aren't included, as the values are in chronological order. If all the files cover the same time interval, the actual years are irrelevant, though you should include them in the filename for good bookkeeping.

The Code

Name your retirement nest-egg simulator *nest_egg_mcs.py*. You'll need the text files described in "**Finding Historical Data**" on **page 249**. Download these files from **https://www.nostarch.com/impracticalpython/** and keep them in the same folder as *nest_egg_mcs.py*.

Importing Modules and Defining Functions to Load Data and Get User Input

Listing 12-1 imports modules and defines a function to read the historical returns and inflation data, as well as another function to get the user's input. Feel free to alter or add to the historical data to conduct experiments after the program is up and running.

nest_egg_mcs.py, part 1

```
    import sys
    import random
❶ import matplotlib.pyplot as plt


❷ def read_to_list(file_name):
```

```
        """Open a file of data in percent, convert to decimal & return a list."""
        ❸ with open(file_name) as in_file:
            ❹ lines = [float(line.strip()) for line in in_file]
            ❺ decimal = [round(line / 100, 5) for line in lines]
            ❻ return decimal

❼ def default_input(prompt, default=None):
        """Allow use of default values in input."""
❽      prompt = '{} [{}]: '.format(prompt, default)
❾      response = input(prompt)
❿      if not response and default:
            return default
        else:
            return response
```

Listing 12-1: Imports modules and defines functions to load data and get user input

The `import` statements should all be familiar. The `matplotlib` library is needed to build the bar chart of results. You only need the plotting functionality, as specified in the `import` statement ❶.

Next, define a function called `read_to_list()` to load a data file and process its contents ❷. You'll pass it the name of the file as an argument.

Open the file using `with`, which will automatically close it ❸, and then use list comprehension to build a list of the contents ❹. Immediately convert the list items, which are in percents, to decimal values rounded to five decimal places ❺. Historical returns are usually presented to two decimal places at most, so rounding to five should be sufficient. You may notice more accuracy in the values of some of the data files used here, but that's just the result of some preprocessing in Excel. End by returning the `decimal` list ❻.

Now, define a function called `default_input()` to get the user's input ❼. The function takes a prompt and a default value as arguments. The prompt and default will be specified when the function is called, and the program will display the default in brackets ❽. Assign a `response` variable to the

user's input ❾. If the user enters nothing and a default value exists, the default value is returned; otherwise, the user's response is returned ❿.

Getting the User Input

Listing 12-2 loads the data files, maps the resulting lists to simple names using a dictionary, and gets the user's input. The dictionary will be used to present the user with multiple choices for an investment type. Overall, user input consists of:

- The type of investment to use (stocks, bonds, or a blend of both)
- The starting value of their retirement savings
- A yearly withdrawal, or spending, amount
- The minimum, most likely, and maximum number of years they expect to live in retirement
- The number of cases to run

nest_egg_mcs.py, part 2

```
    # load data files with original data in percent form
❶ print("\nNote: Input data should be in percent, not decimal!\n")
    try:
        bonds = read_to_list('10-yr_TBond_returns_1926-2013_pct.txt')
        stocks = read_to_list('SP500_returns_1926-2013_pct.txt')
        blend_40_50_10 = read_to_list('S-B-C_blend_1926-2013_pct.txt')
        blend_50_50 = read_to_list('S-B_blend_1926-2013_pct.txt')
        infl_rate = read_to_list('annual_infl_rate_1926-2013_pct.txt')
    except IOError as e:
        print("{}. \nTerminating program.".format(e), file=sys.stderr)
        sys.exit(1)

    # get user input; use dictionary for investment-type arguments
❷ investment_type_args = {'bonds': bonds, 'stocks': stocks,
                          'sb_blend': blend_50_50, 'sbc_blend': blend_40_50_10}

❸ # print input legend for user
    print("   stocks = SP500")
    print("    bonds = 10-yr Treasury Bond")
    print(" sb_blend = 50% SP500/50% TBond")
```

```
    print("sbc_blend = 40% SP500/50% TBond/10% Cash\n")
    print("Press ENTER to take default value shown in [brackets]. \n")

    # get user input
❹ invest_type = default_input("Enter investment type: (stocks, bonds, sb_blend,"
                             " sbc_blend): \n", 'bonds').lower()
❺ while invest_type not in investment_type_args:
        invest_type = input("Invalid investment. Enter investment type " \
                            "as listed in prompt: ")

    start_value = default_input("Input starting value of investments: \n", \
                                '2000000')
❻ while not start_value.isdigit():
        start_value = input("Invalid input! Input integer only: ")

❼ withdrawal = default_input("Input annual pre-tax withdrawal" \
                             " (today's $): \n", '80000')
    while not withdrawal.isdigit():
        withdrawal = input("Invalid input! Input integer only: ")

    min_years = default_input("Input minimum years in retirement: \n", '18')
    while not min_years.isdigit():
        min_years = input("Invalid input! Input integer only: ")

    most_likely_years = default_input("Input most-likely years in retirement: \n",
                                      '25')
    while not most_likely_years.isdigit():
        most_likely_years = input("Invalid input! Input integer only: ")

    max_years = default_input("Input maximum years in retirement: \n", '40')
    while not max_years.isdigit():
        max_years = input("Invalid input! Input integer only: ")

    num_cases = default_input("Input number of cases to run: \n", '50000')
    while not num_cases.isdigit():
        num_cases = input("Invalid input! Input integer only: ")
```

Listing 12-2: Loads data, maps choices to lists, and gets user input

After printing a warning that input data should be in percent form, use the `read_to_list()` function to load the six data files ❶. Use `try` when opening the files to catch exceptions related to missing files or incorrect filenames. Then handle the exceptions with an `except` block. If you need a refresher on `try` and `except`, refer to "**Handling Exceptions When Opening Files** on **page 21**.

The user will have a choice of investment vehicles to test. To allow them to type in simple names, use a dictionary to map the names to the lists of data you just loaded ❷. Later, you'll pass a function this dictionary and its key to serve as an argument: `montecarlo(investment_type_args[invest_type])`. Before asking for input, print a legend to aid the user ❸.

Next, get the user's investment choice ❹. Use the `default_input()` function and list the names of the choices, which are mapped back to the data lists. Set the default to `'bonds'`, in order to see how this supposedly "safe" choice performs. Be sure to tack on the `.lower()` method in case the user accidently includes an uppercase letter or two. For other possible input errors, use a `while` loop to check the input versus the names in the `investment_type_args` dictionary; if you don't find the input, prompt the user for a correct answer ❺.

Continue gathering the input and use the default values to guide the user to reasonable inputs. For example, $80,000 is 4 percent of the starting value of $2,000,000; also, 25 years is a good most-likely value for women entering retirement at 60, a maximum value of 40 will allow them to reach 100, and 50,000 cases should quickly yield a good estimate of the probability of ruin.

For numerical inputs, use a `while` loop to check that the input is a digit, just in case the user includes a dollar sign ($) or commas in the number ❻. And for the `withdrawal` amount, use the prompt to guide the user by letting them know to input *today's* dollars and not to worry about inflation ❼.

Checking for Other Erroneous Input

Listing 12-3 checks for additional input errors. The order of the minimum, most likely, and maximum years in retirement should be logical, and a maximum length of 99 years is enforced. Allowing for a long time

in retirement permits the optimistic user to evaluate cases where medical science makes significant advances in anti-aging treatments!

nest_egg_mcs.py, part 3

```
    # check for other erroneous input
❶ if not int(min_years) < int(most_likely_years) < int(max_years) \
        or int(max_years) > 99:
   ❷ print("\nProblem with input years.", file=sys.stderr)
      print("Requires Min < ML < Max with Max <= 99.", file=sys.stderr)
      sys.exit(1)
```

Listing 12-3: Checks for errors and sets limits in the retirement years input

Use a conditional to ensure that the minimum years input is less than the most likely, the most likely is less than the maximum, and the maximum does not exceed 99 ❶. If a problem is encountered, alert the user ❷, provide some clarifying instructions, and exit the program.

Defining the Monte Carlo Engine

Listing 12-4 defines the first part of the function that will run the Monte Carlo simulation. The program uses a loop to run through each case, and the number of years in retirement inputs will be used to sample the historical data. For both the returns and inflation lists, the program chooses a starting year, or index, at random. The number of years in retirement, assigned to a `duration` variable, is drawn from a triangular distribution built from the user's input. If 30 is chosen, then 30 is added to this starting index to create the ending index. The random starting year will determine the retiree's financial fortunes for the rest of their life! Timing is everything, as they say.

nest_egg_mcs.py, part 4

```
❶ def montecarlo(returns):
       """Run MCS and return investment value at end-of-plan and bankrupt count.'
   ❷ case_count = 0
      bankrupt_count = 0
```

```
        outcome = []

    ❸ while case_count < int(num_cases):
          investments = int(start_value)
      ❹ start_year = random.randrange(0, len(returns))
      ❺ duration = int(random.triangular(int(min_years), int(max_years),
                                         int(most_likely_years)))
      ❻ end_year = start_year + duration
      ❼ lifespan = [i for i in range(start_year, end_year)]
          bankrupt = 'no'

          # build temporary lists for each case
      ❽ lifespan_returns = []
          lifespan_infl = []
          for i in lifespan:
           ❾ lifespan_returns.append(returns[i % len(returns)])
              lifespan_infl.append(infl_rate[i % len(infl_rate)])
```

Listing 12-4: Defines the Monte Carlo function and starts a loop through cases

The `montecarlo()` function takes a `returns` list as an argument ❶. The first step is to start a counter to keep track of which case is being run ❷. Remember that you don't need to use actual dates; the first year in the lists is index 0, rather than 1926. In addition, start a counter for the number of cases that run out of money early. Then start an empty list to hold the outcomes of each run, which will be the amount of money remaining at the end of the run.

Begin the `while` loop that will run through the cases ❸. Assign a new variable, called `investments`, to the starting investment value that the user specified. Since the `investments` variable will change constantly, you need to preserve the original input variable to reinitialize each case. And since all the user input came in as strings, you need to convert the values to integers before using them.

Next, assign a `start_year` variable and pick a value at random from the range of available years ❹. To get the time spent in retirement for this

simulated life, use the `random` module's `triangular()` method to draw from a triangular distribution defined by the user's `min_years`, `most_likely_years`, and `max_years` inputs ❺. According to the documentation, `triangular()` returns a random floating-point number *N* such that low <= *N* <= high and with the specified *mode* between those bounds.

Add this `duration` variable to the `start_year` variable and assign the result to an `end_year` variable ❻. Now, make a new list, named `lifespan`, that captures all the indexes between the starting year and the ending year ❼. These indexes will be used to match the retirement period to the historical data. Next, assign a `bankrupt` variable to `'no'`. Bankrupt means you've run out of money, and later this outcome will end the `while` loop early, using a `break` statement.

Use two lists to store the applicable returns and inflation data for the chosen `lifespan` ❽. Populate these lists using a `for` loop that uses each item in `lifespan` as the index for the returns and inflation lists. If the `lifespan` index is out of range compared to the other lists, use the modulo (`%`) operator to wrap the indexes ❾.

Let's cover a little more background on this listing. The randomly selected `start_year` variable and the calculated `end_year` variable determine how the returns and inflation lists are sampled. The sample is a continuous piece of financial history and constitutes a case. Selecting an *interval* at random distinguishes this program from online calculators that select individual *years* at random and may use a *different year* for each asset class and inflation! Market results aren't pure chaos; bull and bear markets are cyclic, as are inflationary trends. The same events that cause stocks to decline also impact the price of bonds and the rate of inflation. Picking years at random ignores this interdependency and disrupts known behaviors, leading to unrealistic results.

In **Figure 12-7**, the retiree (known as Case 1) has chosen to retire in 1965—at the start of the Great Inflation—and to invest in bonds. Because the end year occurs before the end of the returns list, the retirement span fits nicely in the list. Both returns and inflation are sampled over the same interval.

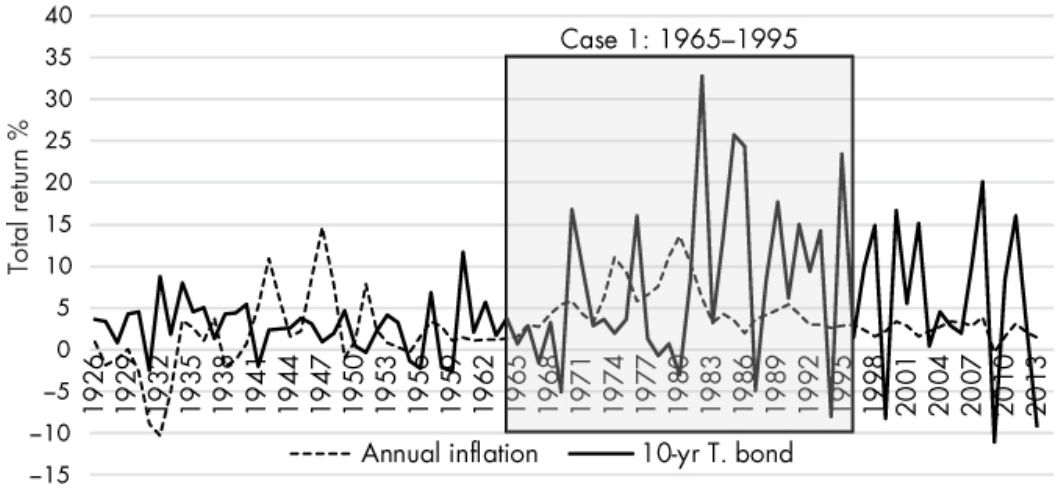

Figure 12-7: Graph of bond and inflation lists, with a retirement starting in 1965 annotated

In **Figure 12-8**, the retiree, or Case 2, has chosen to retire in 2000. Because the list ends in 2013, the 30-year sample taken by the MCS function must "wrap around" and cover years 1926 through 1941. This forces the retiree to endure two recessions and a depression.

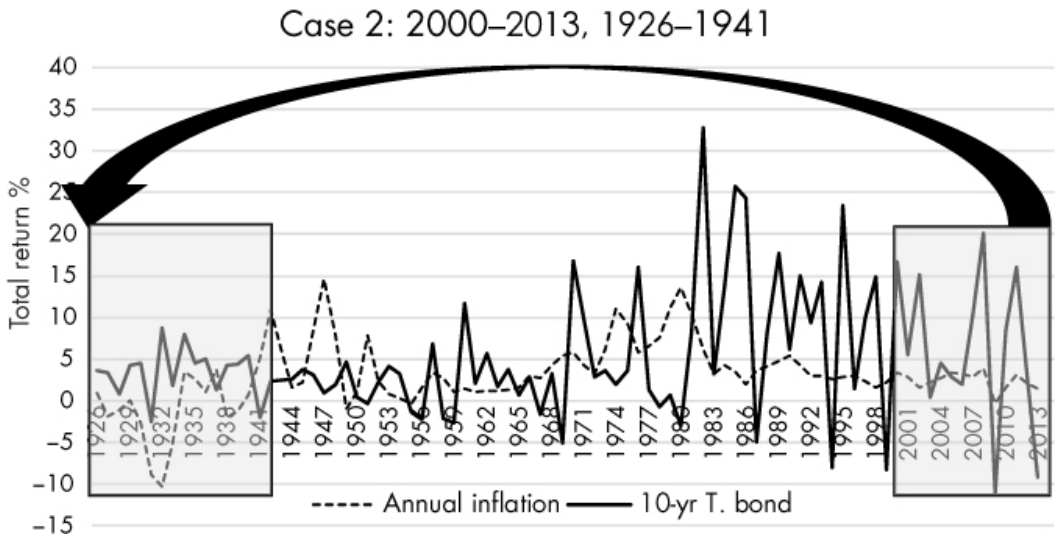

Figure 12-8: Graph of bonds and inflation lists, with segments used in Case 2 annotated

The program will need to simulate the wrap-around segment you see in Case 2—hence the use of the modulo operator, which allows you to treat the lists as endless loops.

Simulating Each Year in a Case

Listing 12-5 continues the `montecarlo()` function and loops through each year of retirement for a given case, increasing or decreasing the investment value based on the returns for that year, subtracting the inflation-

adjusted withdrawal amount from the investments, and checking whether the investments are exhausted. The program saves the final investment value—representing the savings remaining at death—to a list so that the overall probability of ruin can be calculated at the end.

nest_egg_mcs.py, part 5

```
        # loop through each year of retirement for each case run
        ❶ for index, i in enumerate(lifespan_returns):
            ❷ infl = lifespan_infl[index]

            ❸ # don't adjust for inflation the first year
               if index == 0:
                   withdraw_infl_adj = int(withdrawal)
               else:
                   withdraw_infl_adj = int(withdraw_infl_adj * (1 + infl))

            ❹ investments -= withdraw_infl_adj
               investments = int(investments * (1 + i))

            ❺ if investments <= 0:
                   bankrupt = 'yes'
                   break

        ❻ if bankrupt == 'yes':
               outcome.append(0)
               bankrupt_count += 1
           else:
               outcome.append(investments)

        ❼ case_count += 1

    ❽ return outcome, bankrupt_count
```

Listing 12-5: Simulates results for each year in retirement (per case)

Start the `for` loop that will run through all the years in a case ❶. Use `enumerate()` on the `returns` list and use the index that `enumerate()` produced to get the

year's average inflation value from the inflation list ❷. Use a conditional to start applying inflation after the first year ❸. This will slowly increase or decrease the withdrawal amount over time, depending on whether you're in inflationary or deflationary times.

Subtract the inflation-adjusted withdrawal value from the `investments` variable, then adjust `investments` for the year's returns ❹. Check that the value of `investments` is greater than 0. If it's not, set the `bankrupt` variable to `'yes'` and end the loop ❺. For a bankrupt case, append 0 to the `outcome` list ❻. Otherwise, the loop continues until the duration of retirement is reached, so you append the remaining value of `investments` to `outcome`.

A human life has just ended: 30 to 40 years' worth of vacations, grandkids, bingo games, and illnesses gone in much less than a second. So, advance the case counter before looping through the next lifetime ❼. End the function by returning the `outcome` and `bankrupt_count` variables ❽.

Calculating the Probability of Ruin

Listing 12-6 defines a function that calculates the probability of running out of money, also known as the "probability of ruin." If you're risk averse or want to leave a sizeable sum to your heirs, you probably want this number to be less than 10 percent. Those with a greater appetite for risk may be content with up to 20 percent or more. After all, you can't take it with you!

nest_egg_mcs.py, part 6

```
❶ def bankrupt_prob(outcome, bankrupt_count):
      """Calculate and return chance of running out of money & other stats."""
   ❷ total = len(outcome)
   ❸ odds = round(100 * bankrupt_count / total, 1)

   ❹ print("\nInvestment type: {}".format(invest_type))
     print("Starting value: ${:,}".format(int(start_value)))
     print("Annual withdrawal: ${:,}".format(int(withdrawal)))
     print("Years in retirement (min-ml-max): {}-{}-{}"
           .format(min_years, most_likely_years, max_years))
     print("Number of runs: {:,}\n".format(len(outcome)))
```

```
        print("Odds of running out of money: {}%\n".format(odds))
        print("Average outcome: ${:,}".format(int(sum(outcome) / total)))
        print("Minimum outcome: ${:,}".format(min(i for i in outcome)))
        print("Maximum outcome: ${:,}".format(max(i for i in outcome)))

    ❺ return odds
```

Listing 12-6: Calculates and displays the "probability of ruin" and other statistics

Define a function called `bankrupt_prob()` that takes the `outcome` list and `bankrupt_count` variable returned from the `montecarlo()` function as arguments ❶. Assign the length of the outcome list to a variable named `total` ❷. Then, calculate the probability of running out of money as a percentage rounded to one decimal place by dividing the number of bankrupt cases by the total number of cases ❸.

Now, display the input parameters and results of the simulation ❹. You saw an example of this text printout in "**A Qualitative Way to Present Results**" on **page 246**. Finish by returning the `odds` variable ❺.

Defining and Calling the main() Function

Listing 12-7 defines the `main()` function that calls the `montecarlo()` and `bankrupt_count()` functions and creates the bar chart display. The outcomes for the various cases can have a large variance—sometimes you go broke, and other times you're a multimillionaire! If the printed statistics don't make this clear, the bar chart certainly will.

nest_egg_mcs.py, part 7

```
❶ def main():
        """Call MCS & bankrupt functions and draw bar chart of results."""
    ❷ outcome, bankrupt_count = montecarlo(investment_type_args[invest_type])
        odds = bankrupt_prob(outcome, bankrupt_count)

    ❸ plotdata = outcome[:3000]  # only plot first 3000 runs
```

```
    ❹ plt.figure('Outcome by Case (showing first {} runs)'.format(len(plotdata))
                 figsize=(16, 5))  # size is width, height in inches
    ❺ index = [i + 1 for i in range(len(plotdata))]
    ❻ plt.bar(index, plotdata, color='black')
       plt.xlabel('Simulated Lives', fontsize=18)
       plt.ylabel('$ Remaining', fontsize=18)
    ❼ plt.ticklabel_format(style='plain', axis='y')
    ❽ ax = plt.gca()
       ax.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, loc: "{:,}'
                                                            .format(int(x))))
       plt.title('Probability of running out of money = {}%'.format(odds),
                 fontsize=20, color='red')
    ❾ plt.show()

  # run program
❿ if __name__ == '__main__':
      main()
```

Listing 12-7: Defines and calls the `main()` function

Define a `main()` function, which requires no arguments ❶, and immediately call the `montecarlo()` function to get the `outcome` list and the `bankrupt_count()` function ❷. Use the dictionary mapping of investment names to returns lists that you made in **Listing 12-2**. The argument you pass to the `montecarlo()` function is the dictionary name, `investment_type_args`, with the user's input, `invest_type`, as the key. Feed the returned values to the `bankrupt_prob()` function to get the odds of running out of money.

Assign a new variable, `plotdata`, to the first 3,000 items in the `outcome` list ❸. The bar chart can accommodate many more items, but displaying them will be slow as well as unnecessary. As the results are stochastic, you won't gain a lot of extra information by showing more cases.

Now you'll use `matplotlib` to create and show the bar chart. Start by making a plot figure ❹. The text entry will be the title of the new window. The `figsize` parameter is the width and height of the window, in inches. You can scale this by adding a dots-per-inch argument, such as `dpi=200`.

Next, use list comprehension to build indexes, starting with 1 for year one, based on the length of the `plotdata` list ❺. The x-axis location of each vertical bar will be defined by the index, and the height of each bar will be the corresponding `plotdata` item, which represents the money remaining at the end of each simulated life. Pass these to the `plt.bar()` method and set the bar color to black ❻. Note that there are additional display options for bars, such as changing the color of the bar outline (`edgecolor='black'`) or its thickness (`linewidth=0`).

Provide labels for the x- and y-axes and set the font size to 18. Outcomes can reach into the millions, and by default, `matplotlib` will use scientific notation when it annotates the y-axis. To override this, call the `ticklabel_format()` method and set the y-axis style to `'plain'` ❼. This takes care of scientific notation, but there are no thousand separators, making the numbers hard to read. To fix this, first get the current axes using `plt.gca()` ❽. Then, in the next line, get the y-axis and use the `set_major_formatter()` and `Func_Formatter()` methods and a lambda function to apply Python's string-formatting technique for comma separators.

For the plot's title, display the probability of running out of money—captured in the `odds` variable—with a large font in eye-catching red. Then draw the chart to the screen with `plt.show()` ❾. Back in the global space, finish with the code that allows the program to be imported as a module or run in stand-alone mode ❿.

Using the Simulator

The *nest_egg_mcs.py* program greatly simplifies the complex world of retirement planning, but don't hold that against it. Simple models add value by challenging assumptions, raising awareness, and focusing questions. It's easy to get lost in the details of retirement planning—or any complex issue—so you're better off starting with a lay of the land.

Let's work an example that assumes a starting value of $2,000,000, a "safe and secure" bond portfolio, a 4 percent withdrawal rate (equal to $80,000 per year), a 29-30-31 retirement range, and 50,000 cases. If you run this scenario, you should get results similar to those in **Figure 12-9**. You run out of money almost half the time! Because of relatively low yields, bonds

can't keep pace with inflation—remember, you can't blindly apply the 4 Percent Rule, because your asset allocation matters.

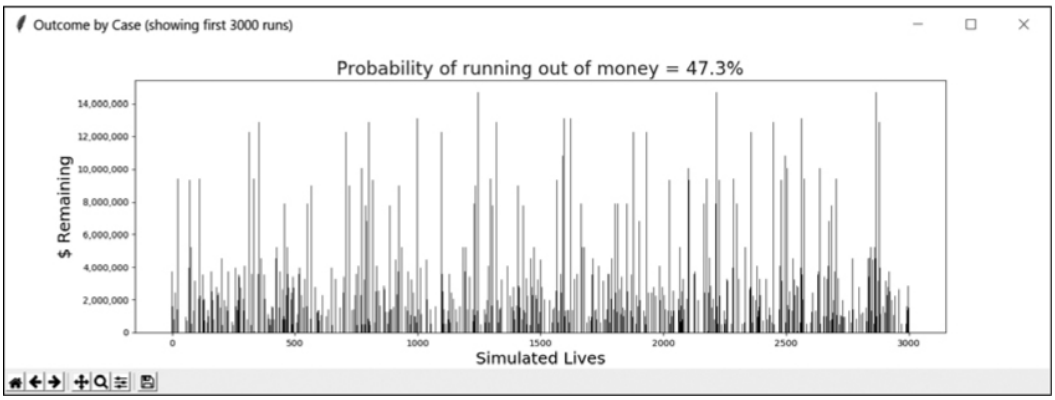

Figure 12-9: Bar chart made with `matplotlib` to represent a Monte Carlo simulation of a bond-only portfolio

Note that the $80,000 withdrawal is *pretax*. Assuming a 25 percent effective tax rate, this leaves a net income of only $60,000. According to the Pew Research Center, the median disposable (after-tax) income of the American middle class is currently $60,884, so you're hardly living high on the hog, despite being a millionaire. If you want $80,000 in *disposable* income, you must divide by 1 minus the effective tax rate; in this case, that's $80,000 / (1 – 0.25) = $106,667. This requires withdrawing slightly over 5 percent per year, with a 20 to 70 percent chance of going broke, depending on the investment type!

Table 12-1 records the results of varying the previous scenario for asset type and withdrawal rate. Results that would be widely considered safe are shaded in gray. If you avoid an all-bond portfolio, the 4 Percent Rule holds up well. Above 4 percent, the growth potential of stocks offers the best chance—of the options considered—to reduce the probability of ruin, and with less risk than most people assume. That's why financial advisers recommend including a healthy dose of stocks in your retirement portfolio.

Table 12-1: Probability of Ruin by Asset Type and Withdrawal Rate for 30-Year Retirement

Asset type	Annual (pretax) withdrawal percentage			
	3%	4%	5%	6%
10-year Treasury bond	0.135	0.479	0.650	0.876
S&P 500 stocks	0	0.069	0.216	0.365
50/50 blend	0	0.079	0.264	0.466
40/50/10 blend	0	0.089	0.361	0.591

Financial advisers also advise against overdoing it in the early years of retirement. A few cruises for the extended family, a posh new house, or an expensive new hobby can push you over the cliff in your later years. To investigate this, copy *nest_egg_mcs.py* and name the copy *nest_egg_mcs_1st_5yrs.py*; adjust the code as described in **Listings 12-8**, **12-9**, and **12-10**:

nest_egg_mcs_1st_5yrs.py, part 1

```
    start_value = default_input("Input starting value of investments: \n", \
                                '2000000')
    while not start_value.isdigit():
        start_value = input("Invalid input! Input integer only: ")

❶ withdrawal_1 = default_input("Input annual pre-tax withdrawal for " \
                               "first 5 yrs(today's $): \n", '100000')
    while not withdrawal_1.isdigit():
        withdrawal_1 = input("Invalid input! Input integer only: ")

❷ withdrawal_2 = default_input("Input annual pre-tax withdrawal for " \
                               "remainder (today's $): \n", '80000')
    while not withdrawal_2.isdigit():
        withdrawal_2 = input("Invalid input! Input integer only: ")
```

```
    min_years = default_input("Input minimum years in retirement: \n", '18')
```

Listing 12-8: Breaks the user's withdrawal input into two parts

In the user-input section, replace the original `withdrawal` variable with two withdrawal variables, and edit the prompt to request the first five years in the first ❶ and the remainder of retirement in the second ❷. Set the defaults to indicate that the user should expect higher withdrawals in the first five years. Include the `while` loops that validate user input.

In the `montecarlo()` function, change the code that adjusts the withdrawal amount for inflation.

nest_egg_mcs_1st_5yrs.py, part 2

```
            # don't adjust for inflation the first year
            if index == 0:
             ❶ withdraw_infl_adj_1 = int(withdrawal_1)
             ❷ withdraw_infl_adj_2 = int(withdrawal_2)
            else:
             ❸ withdraw_infl_adj_1 = int(withdraw_infl_adj_1 * (1 + infl))
             ❹ withdraw_infl_adj_2 = int(withdraw_infl_adj_2 * (1 + infl))

        ❺ if index < 5:
             ❻ withdraw_infl_adj = withdraw_infl_adj_1
            else:
                withdraw_infl_adj = withdraw_infl_adj_2

            investments -= withdraw_infl_adj
            investments = int(investments * (1 + i))
```

Listing 12-9: Adjusts the two withdrawal variables for inflation and determines which to use

Set the inflation-adjusted withdrawal equal to the input withdrawal for just the first year ❶❷. Otherwise, adjust both for inflation ❸❹. This way,

the second withdrawal amount will be "ready" when you switch to it after five years.

Use a conditional to specify when to apply each inflation-adjusted withdrawal ❺. Assign these to the existing `withdraw_infl_adj` variable so you won't have to alter any more code ❻.

Finally, update the printed statistics in the `bankrupt_prob()` function to include the new withdrawal values, as shown in **Listing 12-10**. These should replace the old withdrawal print statement.

nest_egg_mcs_1st_5yrs.py, part 3

```
    print("Annual withdrawal first 5 yrs: ${:,}".format(int(withdrawal_1)))
    print("Annual withdrawal after 5 yrs: ${:,}".format(int(withdrawal_2)))
```

Listing 12-10: Prints the withdrawal values for the two withdrawal periods

You can now run the new experiment (see **Table 12-2**).

Table 12-2: Probability of Ruin by Asset Type and Multiple Withdrawal Rates for 30-Year Retirement

Asset allocation	**Annual (pretax) withdrawal percentage (first five years / remainder)**			
	4% / 4%	**5% / 4%**	**6% / 4%**	**7% / 4%**
10-year Treasury bond	0.479	0.499	0.509	0.571
S&P 500 stocks	0.069	0.091	0.116	0.194
50/50 blend	0.079	0.115	0.146	0.218
40/50/10 blend	0.089	0.159	0.216	0.264

Safe outcomes are shaded in gray in **Table 12-2**, and the first column repeats the constant 4 percent results, as a control. With enough stocks in your portfolio, you can tolerate some early spending, and for this reason, some advisers replace the 4 Percent Rule with either the 4.5 Percent or 5 Percent Rule. But if you retire early—say, between age 55 and 60—your risk of going broke will be greater, whether you experience any high-spending years or not.

If you run the simulator for the 50/50 blend of stocks and bonds, using different years in retirement, you should get results similar to those in **Table 12-3**. Only one outcome (shaded gray) has a probability of ruin below 10 percent.

Table 12-3: Probability of Ruin vs. Retirement Duration for 4% Withdrawal Rate (50/50 Stock-Bond Blend)

Years in retirement	4% withdrawal
30	0.079
35	0.103
40	0.194
45	0.216

Running simulations like these forces people to face hard decisions and form realistic plans for a large segment of their lives. While the simulation "sells" assets every year to fund retirement, a better real-life solution is the *guardrail strategy*, where interest and dividends are spent first and a cash reserve is maintained to avoid having to sell assets during market lows. Assuming you can remain disciplined as an investor, this strategy will allow you to stretch your withdrawals a bit beyond what the simulator calculates as safe.

Summary

In this chapter, you wrote a Monte Carlo–based retirement calculator that realistically samples from historical financial data. You also used `matplotlib` to provide an alternative way of viewing the calculator output. While the example used could have been modeled deterministically, if you add more random variables—like future tax rates, Social Security payments, and health care costs—MCS quickly becomes the only practical approach for modeling retirement strategies.

Further Reading

The Intelligent Investor: The Definitive Book on Value Investing, Revised Edition (HarperBusiness, 2006) by Benjamin Graham is considered by many, including billionaire investor Warren Buffet, the greatest book on investing ever written.

Fooled by Randomness: The Hidden Role of Chance in Life and in the Markets, Revised Edition (Random House Trade Paperbacks, 2005) by Nassim Nicholas Taleb is "an engaging look at the history and reasons for our predilection for self-deception when it comes to statistics." It includes discussions on the use of Monte Carlo simulation in financial analysis.

The Black Swan: The Impact of the Highly Improbable, 2nd Edition (Random House Trade Paperbacks, 2010) by Nassim Nicholas Taleb is a "delightful romp through history, economics, and the frailties of human nature" that also includes discussion of the use of Monte Carlo simulation in finance.

You can find an overview of the 4 Percent Rule at **https://www.investopedia.com/terms/f/four-percent-rule.asp**.

Possible exceptions to the 4 Percent Rule are discussed at **https://www.cnbc.com/2015/04/21/the-4-percent-rule-no-longer-applies-for-most-retirees.html**.

You can find historical financial data at the following websites:

- **http://pages.stern.nyu.edu/~adamodar/New_Home_Page/datafile/histretSP.html**
- **http://www.econ.yale.edu/~shiller/data.htm**

- **http://www.moneychimp.com/features/market_cagr.htm**
- **http://www.usinflationcalculator.com/inflation/historical-inflation-rates/**
- **https://inflationdata.com/Inflation/Inflation_Rate/HistoricalInflation.aspx**

Challenge Projects

Become a chartered Certified Financial Analyst (CFA)[1] by completing these challenge projects.

A Picture Is Worth a Thousand Dollars

Imagine you're a CFA and your prospective client, a wealthy Texas oil prospector, doesn't understand your MCS results on his $10 million portfolio. "Tarnation, boy! What kind of durn fool contraption could have me goin' broke in one case and worth 80 million in the next?"

Make it clearer to him by editing the *nest_egg_mcs.py* program so it runs single, 30-year cases using historical intervals that will result in bad and good results, such as starting at the beginning of the Great Depression versus the end of WWII, but only run the extreme cases. For each year in each case, print out the year, the returns rate, the inflation rate, and the outcome. Even better, edit the bar chart display to use each *year's* outcome rather than each *case's* outcome, for a convincing visual explanation.

Mix and Match

Edit *nest_egg_mcs.py* so that the user can generate their own blend of investments. Use the S&P 500, 10-year Treasury bond, and three-month Treasury bill text files that I provided at the start of the chapter and add anything else you like, such as small-cap stocks, international stocks, or even gold. Just remember that the time intervals should be the same in each file or list.

Have the user pick the investment types and percentage of each. Make sure their input adds up to 100 percent. Then create a blended list by weighting and adding the returns for each year. Finish it off by displaying the investment types and percentages at the top of the bar chart display.

Just My Luck!

Edit *nest_egg_mcs.py* to calculate the probability of encountering a Great Depression (1939–1949) or a Great Recession (2007–2009) during a 30-year retirement. You'll need to identify which index numbers in the returns lists correspond to these events, then tally how many times they occur over however many cases are run. Display the results in the shell.

All the Marbles

For a different way to view the results, copy and edit *nest_egg_mcs.py* so that the bar chart displays all the outcomes sorted from smallest to largest.